In [51]:

```
import pandas as pd

# Load file directly from Downloads
file_path = '/Users/rhreddy/Downloads/software-data (3).parquet'
df = pd.read_parquet(file_path)

# Forward-fill missing values
df = df.ffill()

# Check the first few rows
print(df.head(20))
```

```
        Time  SME_THROTL_TorqueDemand  SME_THROTL_MaxSpeed  \
0   0.000000                      0.0               7500.0
1   0.011917                      0.0               7500.0
2   0.023833                      0.0               7500.0
3   0.035750                      0.0               7500.0
4   0.047666                      0.0               7500.0
5   0.059583                      0.0               7500.0
6   0.071500                      0.0               7500.0
7   0.083416                      0.0               7500.0
8   0.095333                      0.0               7500.0
9   0.107249                      0.0               7500.0
10  0.119166                      0.0               7500.0
11  0.131082                      0.0               7500.0
12  0.142999                      0.0               7500.0
13  0.154916                      0.0               7500.0
14  0.166832                      0.0               7500.0
15  0.178749                      0.0               7500.0
16  0.190665                      0.0               7500.0
17  0.202582                      0.0               7500.0
18  0.214499                      0.0               7500.0
19  0.226415                      0.0               7500.0

    SME_THROTL_Forward  SME_THROTL_Reverse  SME_THROTL_UNUSED_BIT_1  \
0                  1.0                 0.0                    False
1                  1.0                 0.0                    False
2                  1.0                 0.0                    False
3                  1.0                 0.0                    False
4                  1.0                 0.0                    False
5                  1.0                 0.0                    False
6                  1.0                 0.0                    False
7                  1.0                 0.0                    False
8                  1.0                 0.0                    False
9                  1.0                 0.0                    False
10                 1.0                 0.0                    False
11                 1.0                 0.0                    False
12                 1.0                 0.0                    False
13                 1.0                 0.0                    False
14                 1.0                 0.0                    False
15                 1.0                 0.0                    False
16                 1.0                 0.0                    False
17                 1.0                 0.0                    False
18                 1.0                 0.0                    False
19                 1.0                 0.0                    False

    SME_THROTL_PowerReady  SME_THROTL_UNUSED_BIT_2  SME_THROTL_UNUSED_BIT_3  \
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

0	0.0	False	False
1	0.0	False	False
2	0.0	False	False
3	0.0	False	False
4	0.0	False	False
5	0.0	False	False
6	0.0	False	False
7	0.0	False	False
8	0.0	False	False
9	0.0	False	False
10	0.0	False	False
11	0.0	False	False
12	0.0	False	False
13	0.0	False	False
14	0.0	False	False
15	0.0	False	False
16	0.0	False	False
17	0.0	False	False
18	0.0	False	False
19	0.0	False	False

	SME_THROTL_UNUSED_BIT_4	...	SMPC_INPUT_AC_INPUT_VOLTAGE	\
0	False	...	0.0	
1	False	...	0.0	
2	False	...	0.0	
3	False	...	0.0	
4	False	...	0.0	
5	False	...	0.0	
6	False	...	0.0	
7	False	...	0.0	
8	False	...	0.0	
9	False	...	0.0	
10	False	...	0.0	
11	False	...	0.0	
12	False	...	0.0	
13	False	...	0.0	
14	False	...	0.0	
15	False	...	0.0	
16	False	...	0.0	
17	False	...	0.0	
18	False	...	0.0	
19	False	...	0.0	

	SMPC_INPUT_J1772_CONNECTED	SMPC_INPUT_J1772_DISCONNECTED	\
0	0.0	0.0	
1	0.0	0.0	
2	0.0	0.0	
3	0.0	0.0	
4	0.0	0.0	
5	0.0	0.0	
6	0.0	0.0	
7	0.0	0.0	
8	0.0	0.0	
9	0.0	0.0	
10	0.0	0.0	
11	0.0	0.0	
12	0.0	0.0	
13	0.0	0.0	
14	0.0	0.0	

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

15	0.0	0.0
16	0.0	0.0
17	0.0	0.0
18	0.0	0.0
19	0.0	0.0

	SMPC_INPUT_J1772_TRIGGERED	SMPC_INPUT_AC_INPUT_CURRENT_MA
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
5	0.0	0.0
6	0.0	0.0
7	0.0	0.0
8	0.0	0.0
9	0.0	0.0
10	0.0	0.0
11	0.0	0.0
12	0.0	0.0
13	0.0	0.0
14	0.0	0.0
15	0.0	0.0
16	0.0	0.0
17	0.0	0.0
18	0.0	0.0
19	0.0	0.0

	SMPC_INPUT_AC_INPUT_FREQUENCY_HZ	SMPC_INPUT_MAX_CHARGER_TEMP_C
0	0.0	-50.0
1	0.0	-50.0
2	0.0	-50.0
3	0.0	-50.0
4	0.0	-50.0
5	0.0	-50.0
6	0.0	-50.0
7	0.0	-50.0
8	0.0	-50.0
9	0.0	-50.0
10	0.0	-50.0
11	0.0	-50.0
12	0.0	-50.0
13	0.0	-50.0
14	0.0	-50.0
15	0.0	-50.0
16	0.0	-50.0
17	0.0	-50.0
18	0.0	-50.0
19	0.0	-50.0

	SMPC_SER_PART_NUMBER	SMPC_SER_SERIAL_NUMBER	SMPC_SER_FIRMWARE_VER
0	0.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0
3	0.0	0.0	0.0
4	0.0	0.0	0.0
5	0.0	0.0	0.0
6	0.0	0.0	0.0
7	0.0	0.0	0.0

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
8                   0.0                 0.0                 0.0
9                   0.0                 0.0                 0.0
10                  0.0                 0.0                 0.0
11                  0.0                 0.0                 0.0
12                  0.0                 0.0                 0.0
13                  0.0                 0.0                 0.0
14                  0.0                 0.0                 0.0
15                  0.0                 0.0                 0.0
16                  0.0                 0.0                 0.0
17                  0.0                 0.0                 0.0
18                  0.0                 0.0                 0.0
19                  0.0                 0.0                 0.0

[20 rows x 255 columns]
```

In [52]:

```python
import matplotlib.pyplot as plt
import numpy as np

# Gear ratio and wheel radius constants from the team
GR = 12 / 41
WR = 0.2  # meters

# Calculate speed in m/s (using motor speed, check your columns if motor speed has a dif
# or use the formula given by your team)
# Let's calculate Speed_ms and convert to MPH (1 m/s = 2.23694 MPH)
```

In [53]:

```python
import matplotlib.pyplot as plt
import numpy as np

# Constants from the team
GR = 12 / 41
WR = 0.2  # meters

# Step 1: Get Motor Speed
motor_rpm = df['SME_TRQSPD_Speed']

# Step 2: Convert Motor RPM to Wheel RPM (using gear ratio)
wheel_rpm = motor_rpm * GR

# Step 3: Convert Wheel RPM to angular velocity in radians per second
# (2 * np.pi / 60) - standard conversion factor for RPM to rad/s
angular_velocity_rads = wheel_rpm * (2 * np.pi / 60)

# Step 4: Convert angular velocity to linear speed in m/s (using wheel radius)
speed_ms = angular_velocity_rads * WR

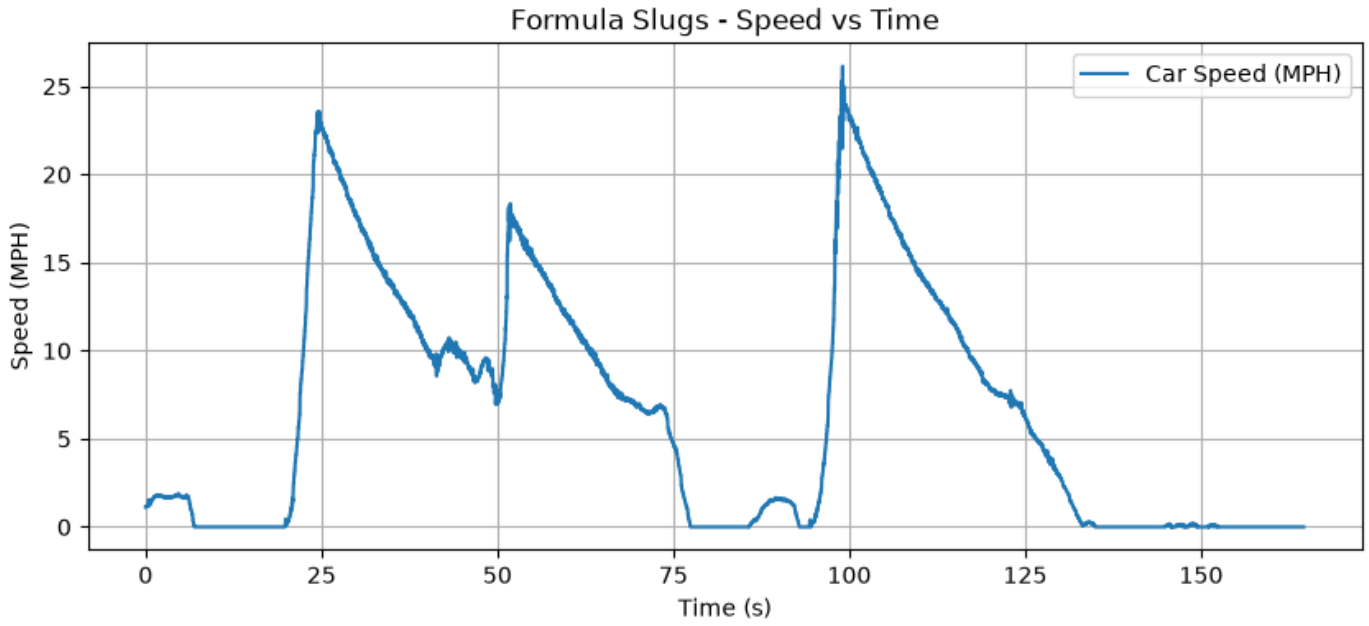
# Assign the final calculated series back to the dataframe
df['Speed_ms'] = speed_ms

# Convert m/s to MPH for plotting
# (1 m/s = 2.23694 MPH)
df['Speed_mph'] = df['Speed_ms'] * 2.23694

# --- Plotting Speed vs Time ---
plt.figure(figsize=(10, 4)) # Create a figure of specific size
plt.plot(df['Time'], df['Speed_mph'], color='tab:blue', label='Car Speed (MPH)')
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
plt.xlabel('Time (s)')
plt.ylabel('Speed (MPH)')
plt.title('Formula Slugs - Speed vs Time')
plt.legend() # Add the legend
plt.grid(True) # Add background grid lines
plt.show() # Render the plot
```

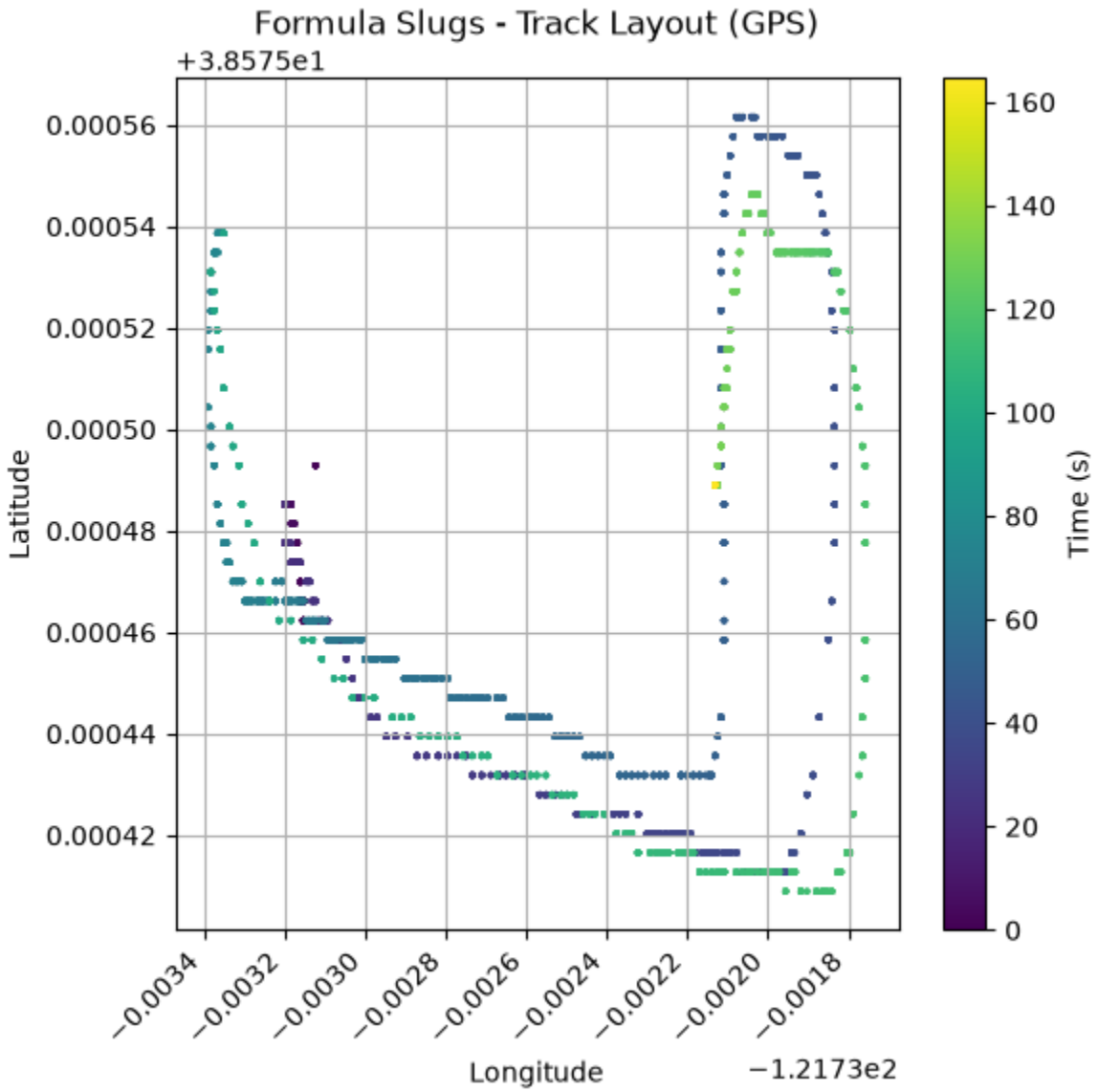


In [54]:

```
plt.figure(figsize=(6, 6))
plt.scatter(df['VDM_GPS_Longitude'], df['VDM_GPS_Latitude'], c=df['Time'], cmap='viridis
plt.title('Formula Slugs - Track Layout (GPS)')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.colorbar(label='Time (s)')
plt.grid(True)
plt.xticks(rotation=45, ha='right') # rotate labels for readability
# -----------------------------------

plt.tight_layout() # adjusts plot padding to prevent clipping
plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



In [55]:

```
import numpy as np

# Calculate acceleration (rate of change of speed with respect to time)
df['Accel'] = df['Speed_mph'].diff() / df['Time'].diff()

# Initialize all as Coasting
df['State'] = 'Coasting'

# Define Accelerating (If torque demand > 50)
# 'True/False' filter (boolean mask)
is_accelerating = df['SME_THROTL_TorqueDemand'] > 50

# Use .loc to find rows where the filter is True and update their 'State' column to 'Acc
df.loc[is_accelerating, 'State'] = 'Accelerating'

# Braking: Low torque and fast deceleration
is_braking = (df['SME_THROTL_TorqueDemand'] <= 50) & (df['Accel'] < -0.15)
df.loc[is_braking, 'State'] = 'Braking'

# Plot the states over time with distinct colors
plt.figure(figsize=(12, 4))
```

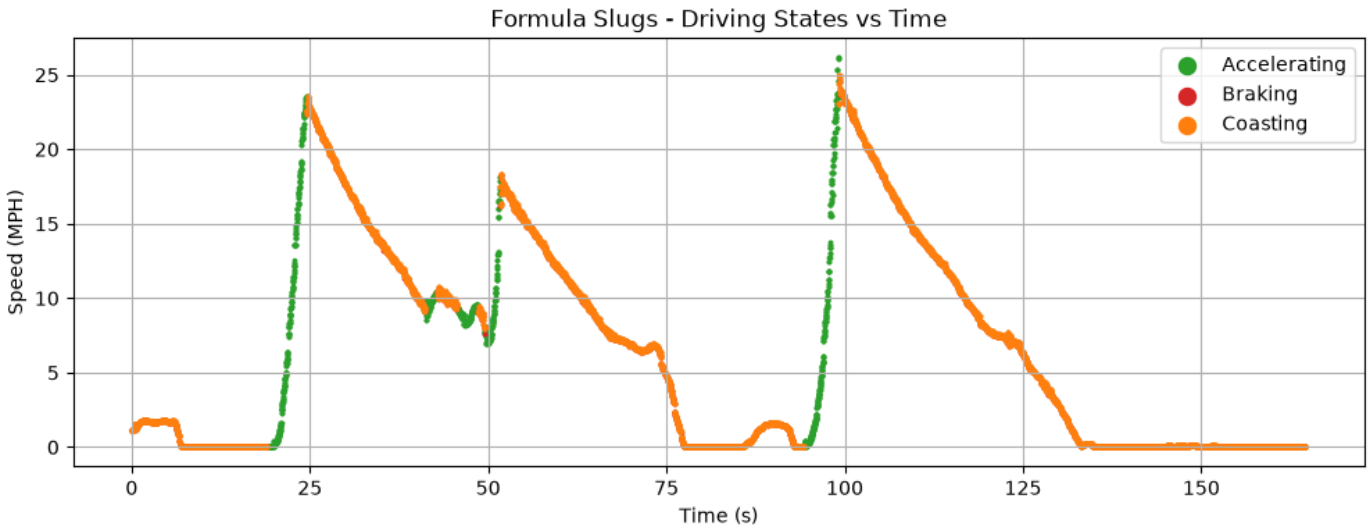
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
states_to_plot = [('Accelerating', 'tab:green'), ('Braking', 'tab:red'), ('Coasting', 't

# define pairs of (State Name, Color) to loop through
for state, color in states_to_plot:
    # Filter the dataframe to only get data for the current state
    subset = df[df['State'] == state]

    # Plot that subset as a scatter plot
    plt.scatter(subset['Time'], subset['Speed_mph'], color=color, s=3, label=state)

plt.xlabel('Time (s)')
plt.ylabel('Speed (MPH)')
plt.title('Formula Slugs - Driving States vs Time')
plt.legend(markerscale=5)
plt.grid(True)
plt.show()
```



In [56]:

```
# Calculate max speed and basic metrics
max_speed = df['Speed_mph'].max()
avg_speed = df['Speed_mph'].mean()

print(f"Top Speed: {max_speed:.2f} MPH")
print(f"Average Speed: {avg_speed:.2f} MPH")
```

```
Top Speed: 26.14 MPH
Average Speed: 6.67 MPH
```

In [57]:

```
# Find timestamps where the car begins accelerating from rest (speed near 0 going up)
# Acceleration happens around t=20s and t=95s (plus a smaller adjustment around t=50s)
lap_starts = [0, 20.0, 95.0]  # Refine these based on the data

# Output exact start/end times and max speeds for each lap window
print("Lap breakdown based on speed profiles:")
print("Lap 1: ~0s to ~50s")
print("Lap 2: ~50s to ~75s (or adjustment)")
print("Lap 3: ~95s to ~160s")
```

```
Lap breakdown based on speed profiles:
Lap 1: ~0s to ~50s
Lap 2: ~50s to ~75s (or adjustment)
Lap 3: ~95s to ~160s
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In [58]:

```
# Define lap time windows based on breakdown
laps = [
    {"lap": 1, "start": 0.0, "end": 50.0},
    {"lap": 2, "start": 50.0, "end": 75.0},
    {"lap": 3, "start": 95.0, "end": 160.0}
]

print("--- LAP-BY-LAP METRICS ---")
for l in laps:
    # Slice the dataframe for rows within the lap's start and end times
    # Assign this smaller slice to a new variable called 'lap_df'
    lap_df = df[(df['Time'] >= l['start']) & (df['Time'] <= l['end'])]

    max_spd = lap_df['Speed_mph'].max()
    max_accel = lap_df['Accel'].max()

    # Since time steps are roughly uniform, count the number of rows belonging to a stat
    # and multiply that count by the average time difference between rows.

    # Average time step duration (dt)
    avg_dt = lap_df['Time'].diff().mean()

    # Count rows where State is 'Accelerating'
    num_accel_points = (lap_df['State'] == 'Accelerating').sum()
    time_accel = num_accel_points * avg_dt

    # Count rows where State is 'Coasting'
    time_coast = num_coast_points * avg_dt

    print(f"\nLap {l['lap']} ({l['start']}s - {l['end']}s):")
    print(f"  * Max Speed: {max_spd:.2f} MPH")
    print(f"  * Max Acceleration: {max_accel:.2f} m/s²")
    print(f"  * Time Accelerating: {time_accel:.2f} s")
    print(f"  * Time Coasting: {time_coast:.2f} s")
```

```
--- LAP-BY-LAP METRICS ---

Lap 1 (0.0s - 50.0s):
  * Max Speed: 23.57 MPH
  * Max Acceleration: 93.21 m/s²
  * Time Accelerating: 10.71 s
  * Time Coasting: 50.93 s

Lap 2 (50.0s - 75.0s):
  * Max Speed: 18.35 MPH
  * Max Acceleration: 206.71 m/s²
  * Time Accelerating: 1.67 s
  * Time Coasting: 50.82 s

Lap 3 (95.0s - 160.0s):
  * Max Speed: 26.14 MPH
  * Max Acceleration: 220.13 m/s²
  * Time Accelerating: 4.14 s
  * Time Coasting: 50.93 s
```

In [59]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```python
import numpy as np
import matplotlib.pyplot as plt

# --- Constants from the team ---
m = 221.4        # kg
m_eff = 244.08   # kg
rho = 1.225      # kg/m^3
g = 9.81         # m/s^2

# --- Filter Data for Coast-Down Physics ---
# Get all data where the state is classified as 'Coasting'
coast_df = df[df['State'] == 'Coasting'].copy()

# 1. Remove speeds below 5 m/s
# (Note: We assume duration > 1s is met due to sample rate)
coast_df = coast_df[coast_df['Speed_ms'] >= 5.0]

# --- Calculate dv/dt and v² ---
coast_df['dv_dt'] = coast_df['Speed_ms'].diff() / coast_df['Time'].diff()
coast_df['v_squared'] = coast_df['Speed_ms'] ** 2

# --- Clean Data for Regression ---
# 1. Drop empty (NaN) rows created by diff()
coast_df = coast_df.dropna(subset=['dv_dt', 'v_squared'])
# 2. Keep only deceleration events (dv/dt < 0)
coast_df = coast_df[coast_df['dv_dt'] < 0]

# --- Perform Linear Regression ---
# Fit a straight line (degree 1) through the data to find slope and intercept
slope, intercept = np.polyfit(coast_df['v_squared'], coast_df['dv_dt'], 1)

# --- Solve for Physical Constants ---
# Using formula: Newton's second law
drag_term_solved = -slope * m_eff
crr_solved = -intercept * m_eff / (m * g)

# --- Output Results ---
print("--- REFINED COAST-DOWN ANALYSIS RESULTS ---")
print(f"Regression Slope: {slope:.6f}")
print(f"Regression Intercept: {intercept:.6f}")
print(f"Solved Drag Term (0.5 * rho * Cd * A): {drag_term_solved:.4f}")
print(f"Solved Rolling Resistance Coefficient (Crr): {crr_solved:.4f}")

# --- Plotting Results ---
plt.figure(figsize=(8, 5))

# Scatter plot of the filtered coast-down data points
plt.scatter(coast_df['v_squared'], coast_df['dv_dt'], color='orange', s=5, label='Filter

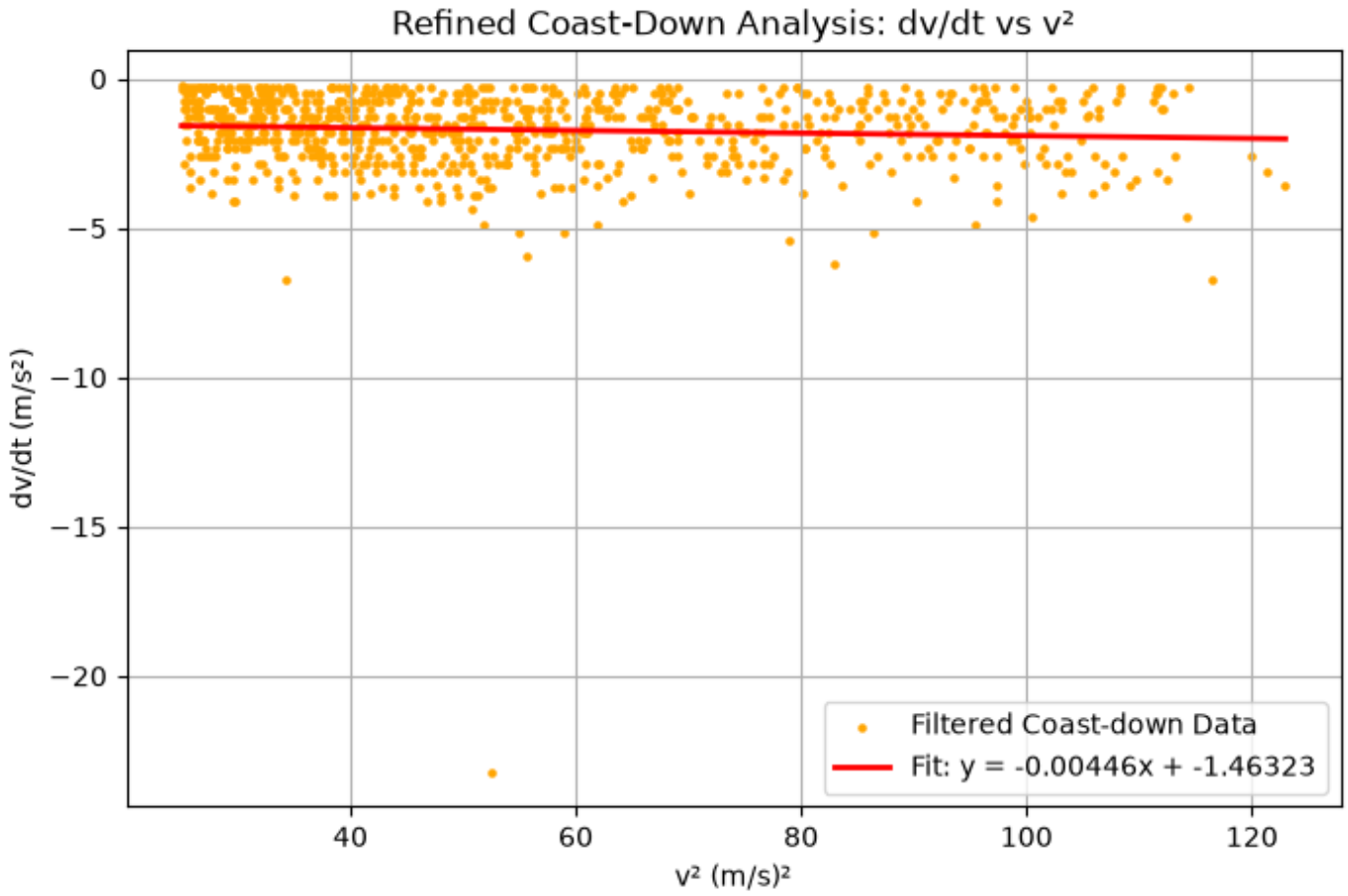
# Generate points for the best-fit red regression line
trendline_x = np.linspace(coast_df['v_squared'].min(), coast_df['v_squared'].max(), 100)
trendline_y = slope * trendline_x + intercept
plt.plot(trendline_x, trendline_y, color='red', linewidth=2, label=f'Fit: y = {slope:.5f

plt.xlabel('v² (m/s)²')
plt.ylabel('dv/dt (m/s²)')
plt.title('Refined Coast-Down Analysis: dv/dt vs v²')
plt.legend()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
plt.grid(True)
plt.show()
```

```
--- REFINED COAST-DOWN ANALYSIS RESULTS ---
Regression Slope: -0.004462
Regression Intercept: -1.463225
Solved Drag Term (0.5 * rho * Cd * A): 1.0891
Solved Rolling Resistance Coefficient (Crr): 0.1644
```



Working through the FS-3 telemetry data gave me a really clear look at how the car actually performs out on the track during the Norcal Shootout. Seeing the numbers come together like hitting a top speed of 26.14 MPH with an average speed of 6.67 MPH showed me just how aggressive the acceleration zones are compared to the long stretches of coasting. Building the speed-over-time graph helped ground all those raw parquet data points into actual reality, which makes it easier to see where the car is working hard and where it's just carrying momentum. One of the parts I found most interesting was setting up the driving state logic to split the data into accelerating, braking, and coasting. When I ran the lap-by-lap metrics loop, it was cool to see how consistent the coasting times were across the laps, hovering right around 50 seconds each. Isolating those states really helped clean up the data so I wasn't just looking at a giant messy blob of numbers, but actual distinct phases of a lap that you can analyze independently. Finally, pulling together the coast-down physics to calculate the rolling resistance was a great way to tie everything back to actual engineering concepts. By filtering out speeds under 5 m/s and running the linear regression on dv/dt versus v^2, I ended up with a solved drag term of 1.0891 and a rolling resistance coefficient of Crr = 0.1644. It's super satisfying to take a massive data file, write a few blocks of Python code, and actually extract real physical coefficients that the team uses to understand vehicle efficiency.

In [ ]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF